

# My Contributions to TML4-Pipeline

---

*A running log of my work on the TML4-Pipeline project — a Streamlit-based machine learning tool that predicts cancer immunotherapy response from gene-expression data, built under Prof. Mark Kon's supervision. Last updated: August 8, 2026.*

---

## Overview of my role

While the team split work across several Git branches — with Angelina, in particular, owning core pipeline logic like the scheduler, feature mapping, and classifier bug fixes — my role has centered on the parts that hold the *whole system* together: reconciling everyone's independent branches into one coherent six-module pipeline, stress-testing it against real data to surface hidden bugs, planning how it eventually gets deployed publicly, and scouting the project's next phase (image data). Below is a chronological account of that work.

---

## 1. Merging five parallel branches into a unified pipeline

The project started as five separate implementations of the same idea, developed independently on the `as`, `mvp`, `master`, `mvp-pipeline`, and `zz` branches. My first major task was reconciling these into the single six-module architecture the professor asked for — Loader → Preprocessor → Feature Map → Classifier → Evaluator → Reporter — so that any teammate's technique could be toggled in as an interchangeable option rather than living in its own silo.

This took a few stages:

- **Cross-walking all five branches.** I had Claude read every branch's source (not just its README, since some, like `master`, didn't document its actual structure) and produce a module cross-walk table showing which branch implemented which of the six canonical stages, and under what names.
- **Building the unifying layer.** Rather than rewriting anyone's code, we used `git subtree add` to import each branch's code verbatim under `contrib/{as,mvp,master}/`, then wrote a thin adapter layer on top: `core/` (shared data model, stage registry, pipeline assembler), `stages/` (adapters registering each branch's techniques into the shared six-stage slots), and `ui/` (Streamlit rendering, dropdowns, optional SHAP explanations). This preserved everyone's original work with attribution while making it swappable.
- **Solving real integration bugs along the way**, including:
  - A `sys.modules` collision where both the `as` and `mvp` branches defined a top-level `pipeline` module, resolved with a custom loader that snapshots and evicts `sys.modules` during import.
  - A binary-vs-multiclass crash in `master`'s evaluator, where KTSP's 2-column probability output broke on multiclass tasks — fixed by gating `y_proba` to binary-only tasks.
  - A PCA crash on small cohorts where `n_components` exceeded `n_samples`, fixed by clamping inside the transformer wrapper.
  - A deployment-breaking issue where a push flattened the package structure (`core/`, `stages/`, `ui/`) into the repo root, silently breaking imports and dropping the `stages/__init__.py` that auto-registers every stage. I diagnosed this and repackaged a corrected zip with git commands to fix it.

- A `starlette/streamlit` import conflict caused by a stale FastAPI dependency left over from the `master` branch — traced to environment reuse rather than the code itself, and fixed via a clean venv rebuild from `requirements.txt`.
- **Later, porting in the `zz` branch's PDF reporting module** (`stages/report_pdf.py`) and its `ZZKTSPClassifier` adapter, deduplicating the resulting UI dropdowns with a `ui/grouping.py` module that collapses identical techniques contributed by multiple branches into a single row (tagged with which branches contribute it), and bundling all eight Tan-format benchmark datasets into a shared catalog so every branch's techniques could be tested against the same data.
- Along the way, I also generated an initial rendering of the project's `.drawio` architecture diagram, color-coded by contributing branch, to give the professor (whose background is math/stats rather than implementation) a clear visual of the module flow. Angelina later improved upon the design.

The test suite grew from **17 to 59 tests** over the course of this work, all developed test-first.

---

## 2. Resolving the binary vs. multiclass architecture question

Partway through the merge, I noticed the `master` branch used a hierarchical K-TSP decision tree built for *multi-class* classification (including the 14-class GCM diagnostic dataset), while the rest of the project's actual goal — predicting immunotherapy response — is inherently binary (responds / doesn't respond). I worked through the distinction, confirmed GCM is a diagnostic dataset unrelated to treatment response, and drafted the email to the professor laying out the binary-vs-multiclass tradeoff so the team could make a deliberate call rather than inheriting it by accident. The team settled on binary-only, and I rebuilt the pipeline's module set (`m1_loader` through `m6_report`, plus `pipeline.py`, `estimators.py`, `app.py`, `sample_data.py`) around that decision, dropping multiclass machinery entirely and documenting why.

Follow-on work in this phase included:

- Fixing a Streamlit session-state bug where local variables were dropped on rerun (moved data/results into `st.session_state`).
  - Making cross-validation (module 5) optional with a fold-count cap based on class size.
  - Hardening the Tan-format file loader to handle gene-ID columns, corner cells, and auto-detected delimiters.
  - Adding a dataset-naming field with auto-suggested names from filenames, and surfacing full run configuration (dataset + every module choice) in both the results panel and the PDF report.
  - Building a Save/Load model tab (session-only, in-memory).
  - Replacing an indeterminate spinner with a stage-level progress bar.
  - Updating the `.drawio` diagram to reflect the finalized six-module design, including the save/load flow and a planned-features section.
- 

## 3. Flagging a security vulnerability before it hit production

While reviewing the Save/Load feature, I identified that deserializing user-uploaded models with `pickle.loads()` is an arbitrary-code-execution risk the moment the app goes public — harmless on localhost, but a real vulnerability once anyone outside the team can upload a file. I documented this in a security section of the README with concrete, safer alternatives (skops as the primary recommendation, ONNX as a secondary option), explicitly noting that superficially similar options like `joblib` or `cloudpickle` do **not** actually fix the underlying issue since they still execute arbitrary code on load. Angelina acted on this

and replaced the format with an encrypted `.tml4model` scheme, which closed the vulnerability ahead of any public deployment.

---

## 4. Building the profiling harness and hunting down real bugs

To validate the pipeline's brute-force "Auto" mode (which exhaustively searches every preprocessor × feature-map × classifier combination), I built a dedicated profiling harness, `profiling/profile_grid.py`, test-first (pytest-compatible, but also runnable standalone), with a companion SGE `qsub` script so it can run on the BU Shared Computing Cluster under the lab's `ds490` allocation, with hard subprocess-based timeouts (default 300s) to reliably kill hung fits from C-extension code.

I ran this harness across all eight benchmark datasets (Colon, DLBCL, GCM, Leukemia, Lung, Prostate1/2/3) over **three separate profiling runs**, each of which surfaced concrete bugs that fed directly back into the team's code:

| Run | What it found                                                                                                                                                                                                                                                                             |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | A PCA-50 failure on small cohorts (fixed by Angelina with AdaptivePCA)                                                                                                                                                                                                                    |
| 2   | An SVM (RBF) near-total collapse — 126 of 128 combinations across every dataset predicting only the majority class, traced to a hardcoded <code>gamma=0.05</code> that turned out to be ~800× too large for ~16,000-gene data                                                             |
| 3   | Verification that Angelina's <code>gamma="scale"</code> fix generalized across <b>all eight datasets</b> — confirmed 0 of 128 collapses post-fix, with SVM (RBF) now a genuine top performer (e.g., 1.00 on Lung, 0.90 on Colon, 0.97 on Leukemia/Prostate3) rather than a degenerate one |

Other findings from this work: a class-conditional-PCA + linear-SVM combination that produced non-convergent, fake majority-class accuracies; and an open, still-unresolved timeout problem where both classic and Hist Gradient Boosting stall on wide datasets like GCM — Hist Gradient Boosting turned out to be *no faster* than classic GB on wide/short data, doubling total grid compute time with no accuracy benefit, which is now flagged as something the grid's cost model needs to account for.

I also used the harness to validate Angelina's LPT parallel scheduler, confirming it delivers real efficiency gains (up to +36 percentage points at 16 cores) — and to check her `combo_cost` scheduling weights, where I found a significant mis-calibration (Hist Gradient Boosting under-weighted by roughly 25×, Spearman rank correlation only 0.42–0.49 against actual runtimes), flagged for the team to re-tune.

Each run produced a written, teammate-facing summary, and for runs 2 and 3 I rewrote the branch README to replace assumption-based claims with evidence-grounded ones — including retracting an earlier "~4× faster" HistGB claim once profiling disproved it, and updating the "SVM (RBF) is permanently degenerate" language once the fix was verified.

**Where this stands now:** a fourth profiling pass is the next step once any further scheduler or classifier changes land, so the README's benchmark table always reflects the pipeline's *actual* current behavior rather than a stale run.

---

## 5. Deployment and infrastructure planning

With the pipeline stabilizing, I took on researching how the tool actually gets in front of users, since the professor wants it publicly available and free. This involved:

- A deployment checklist covering both **Streamlit Community Cloud** and the **BU lab's own server** (Docker + nginx reverse proxy or Caddy with WebSocket upgrade headers for Streamlit, systemd, TLS via certbot), including cross-cutting prerequisites like pinning a `requirements-lock.txt`, declaring a Python version, and adding a research-use disclaimer banner before any public launch.
- A deeper architecture investigation comparing the BU Shared Computing Cluster, a student-managed lab server, dedicated hardware (evaluating options like the ASUS Ascent GX10 / NVIDIA DGX Spark), and a managed cloud alternative (Railway + Supabase).
- Establishing a key technical fact that reshaped the whole conversation: the pipeline is **pure CPU-bound scikit-learn** with no GPU libraries anywhere in the stack, so the brute-force combinatorial grid search is embarrassingly parallel and CPU/RAM-bound rather than GPU-bound — meaning the team's instinct that they needed GPU hardware was a misdiagnosis, and the SCC's GPU queues are largely irrelevant unless the pipeline is later ported to something like RAPIDS/cuML.
- Concluding that the SCC cannot serve as a public-facing backend under BU's Acceptable Use Policy, narrowing the realistic options to a managed backend with a job queue, or the lab server for a bounded, authenticated user base.

This work is documented in `DEPLOYMENT.md` and is still an open decision pending final team sign-off.

---

## 6. Scouting the project's expansion into image data

When the professor assigned the team to identify a public image dataset (paired with a peer-reviewed paper) as a path toward extending the pipeline from gene-expression matrices into image classification, I ran a research pass evaluating six candidate dataset/paper pairs: **Br35H** (brain MRI, binary tumor/no-tumor), **BreakHis** (breast histopathology, binary benign/malignant), a Kaggle **Pancreatic CT** dataset, **LC25000** (lung/colon histopathology), **BUSI** (breast ultrasound), and **HAM10000/PatchCamelyon** (skin lesion imaging).

My recommendation to the team:

- **Br35H** as the primary candidate — small, balanced, genuinely free on Kaggle, and a direct match to the professor's own tumor/no-tumor MRI example.
- **BreakHis** as a strong co-primary — its founding paper (Spanhol et al., 2016) is itself structured as a multi-feature-extractor × multi-classifier meta-pipeline, which conceptually mirrors TML4's own design almost exactly.
- The Kaggle **Pancreatic CT** dataset as a thematically-aligned supplementary option, given our project's pancreatic cancer focus, with the caveat that its provenance is weaker than the other two.

I also flagged practical risks the team needs to watch for before committing: over-optimistic reported accuracies on some sets (especially LC25000 and the pancreatic Kaggle set), patient-level data leakage risk in slice-based datasets like BreakHis, and the distinction between genuinely free primary sources versus paywalled IEEE DataPort mirrors for both Br35H and BreakHis.

---

## Working principles I've tried to hold to throughout

- **Test-driven development.** Every module above went in test-first, with the suite growing from 17 → 59 → 67+ tests as the project matured.
  - **Profile before optimizing.** Every real bug we've caught (SVM gamma, AdaptivePCA, the GB timeout) was found by running the harness against actual benchmark data — not by code review or the existing test suite alone.
  - **Surface warnings, defer decisions to the team.** When I find something (a FutureWarning, a mis-calibrated cost model, a stale README claim), I document it clearly rather than silently fixing it myself if it's a design decision that affects teammates' code.
  - **Verify fixes generalize before declaring victory.** The SVM gamma fix wasn't "done" after one spot-check — it took a full third profiling run across all eight datasets before I updated the README with numbers I could stand behind.
- 

*This document will be updated as I continue contributing — next up: the fourth profiling run once scheduler changes land, and finalizing the deployment path.*